LLMOps: API Cost, Latency, and Prompt Version Management

Name: Nirmal Raj Eganathan
Student Id: 2025AE05579
Date: 05-11-2026
Assignment: Mlops-Assignment 2

1. Introduction

This assignment helps students operationalize an LLM application by measuring latency, calculating token-based costs, and managing prompt versions in MLflow. LLMOps extends MLOps by treating prompts, retrieval context, guardrails, and output quality as versioned production artifacts, while also managing inference cost, latency, and token consumption. developers. OpenAI.

2. Task 1: Latency Measurement

Latency Results Table

```
(venv) PS C:\Users\NEG100\Downloads\MLOps_Assignment_2> python src/task1_latency.py
   question_id prompt_version  model_name                start_time                  end_time  latency_seconds response_status
0            1             v1  gpt-5-mini 2026-11-05 15:37:43.760769 2026-11-05 15:37:52.259213            8.498         success
1            2             v1  gpt-5-mini 2026-11-05 15:37:52.259230 2026-11-05 15:38:03.555254           11.296         success
2            3             v1  gpt-5-mini 2026-11-05 15:38:03.555333 2026-11-05 15:38:09.701962            6.147         success
3            4             v1  gpt-5-mini 2026-11-05 15:38:09.701982 2026-11-05 15:38:16.944078            7.242         success
4            5             v1  gpt-5-mini 2026-11-05 15:38:16.944091 2026-11-05 15:38:25.660342            8.716         success
```

Latency Summary Statistics

```
(venv) PS C:\Users\NEG100\Downloads\MLOps_Assignment_2> python src/latency_summary.py

Latency Summary

Mean   : 8.380
Median : 8.498
Min    : 6.147
Max    : 11.296
Std Dev: 1.931
```

Interpretation

The OpenAI GPT-5 Mini model was accessed through the OpenAI API using credentials stored securely in environment variables. Five academic questions were submitted to the model, and latency was measured for each request. The latency results ranged from 6.147 seconds to 11.296 seconds.

The slowest request required approximately 11.296 seconds. Variations in latency can occur due to network conditions, response length, backend processing time, and temporary workload on the API provider's infrastructure. Despite these differences, latency remained relatively consistent across all requests, indicating stable API performance for academic question-answering workloads.

3. Task 2: Token Usage and Cost Analysis

Token and Cost Results & Monthly cost Projection

```
(venv) PS C:\Users\NEG100\Downloads\MLOps_Assignment_2> python src/task2_tokens_cost.py

Average Cost Per Request: $0.001487
Projected Monthly Cost (10,000 requests): $14.87
   question_id  input_tokens  output_tokens  total_tokens  latency_seconds  cost_per_request
0            1            20            889           909           11.237          0.001783
1            2            19            694           713            9.094          0.001393
2            3            24            651           675            7.887          0.001308
3            4            24            571           595            6.191          0.001148
4            5            19            900           919            7.667          0.001805
```

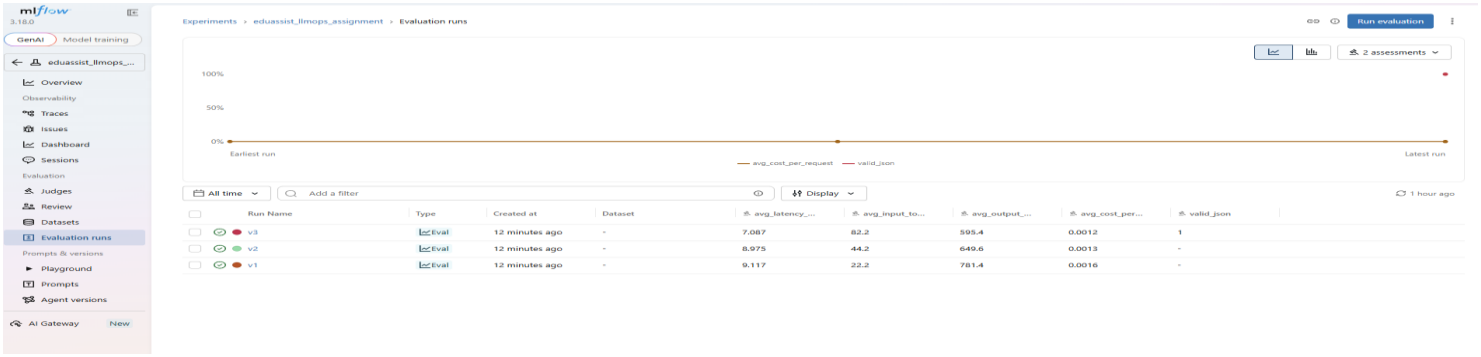
Interpretation

Token usage information was extracted directly from API responses. The input tokens, output tokens, total tokens, and cost per request were computed using published pricing assumptions of $0.25 per million input tokens and $2.00 per million output tokens.

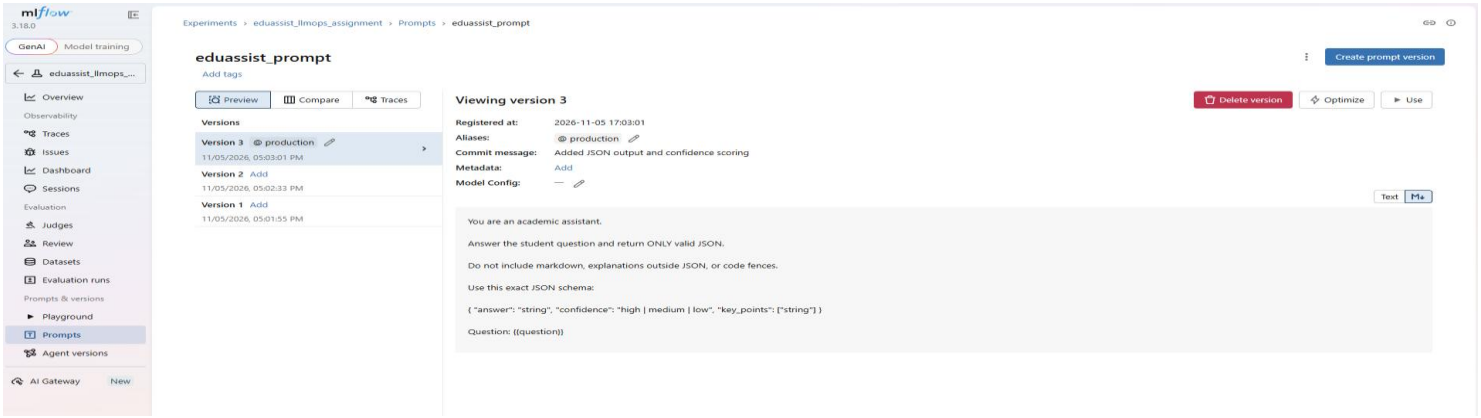
The average cost per request was calculated as: **$0.001487**
Using the formula: Projected Monthly Cost = Average Cost × 10,000 Requests
The projected cost for 10,000 monthly requests was: **$14.87**
A practical cost optimization strategy would be reducing response verbosity and limiting unnecessary output tokens. Since output tokens contribute significantly to total cost, enforcing concise responses can reduce operational expenses. Other optimization approaches include prompt caching, context compression, and model routing strategies that use smaller models for simpler questions while reserving larger models for more complex tasks.

4. Task 3: Prompt Versioning and Evaluation

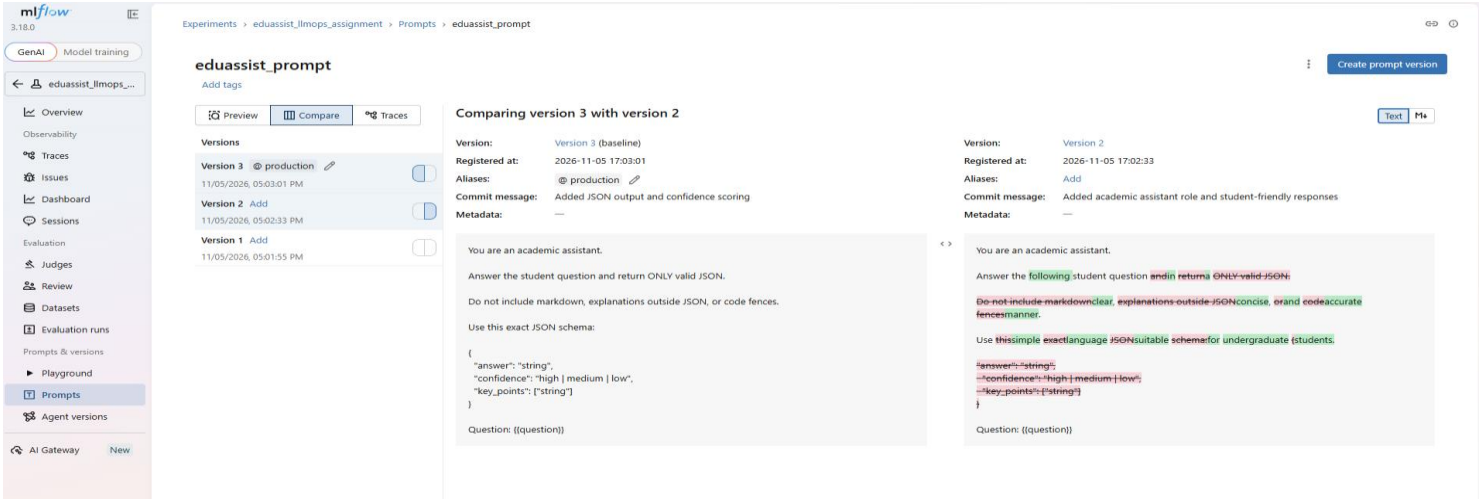
MLflow Experiment Runs (MLflow experiment showing runs v1, v2, and v3 with evaluation metrics)



Prompt Registry (MLflow Prompt Registry showing prompt versions v1, v2, and v3)



Prompt Diff Comparison (Comparison between prompt version 2 and prompt version 3) & (Prompt Version 3 assigned to the production alias.)



Prompt Comparison Table

```
(venv) PS C:\Users\NEG100\Downloads\MLOps_Assignment_2> python src/task3_prompt_eval.py
  prompt_version  avg_latency_seconds  avg_input_tokens  avg_output_tokens  avg_cost_per_request  valid_json
0             v1                9.117              22.2              781.4              0.001568         NaN
1             v2                8.975              44.2              649.6              0.001310         NaN
2             v3                7.087              82.2              595.4              0.001211         1.0
```

Interpretation

Three prompt versions were implemented and evaluated using MLflow.
Version 1 used a basic prompt requesting a clear answer.

Version 2 introduced structured instructions, positioning the model as an academic assistant and specifying undergraduate-level language.
Version 3 required strict JSON output containing an answer, confidence level, and key points.
Prompt Version 3 demonstrated the best overall performance. Although it consumed more input tokens because of the JSON schema instructions, it generated fewer output tokens and produced the lowest cost per request. It also recorded the fastest response time and achieved a JSON validity score of 1.0, indicating perfect compliance with the required output schema.
As a result, Version 3 was promoted to the production alias within the MLflow Prompt Registry.

Production Promotion Justification
Prompt Version 3 was selected and promoted to production after evaluating all versions using latency, token usage, operational cost, output quality, and JSON validity metrics. The prompt provided structured responses while maintaining the best balance between performance and reliability. Although it required more input tokens due to detailed instructions, it generated the fewest output tokens, resulting in the lowest average cost per request of $0.001211.
Version 3 also achieved the fastest average response time of 7.087 seconds, improving the overall user experience. Most importantly, every response generated by Version 3 was valid JSON, resulting in a JSON validity rate of 1.0. This makes the output suitable for downstream automation, storage, analytics, and integration with other systems without requiring additional processing.
The inclusion of confidence scores and key points further enhances usability and transparency. In contrast, Versions 1 and 2 produced free-text outputs, which are more difficult to validate and integrate into automated workflows. Considering operational efficiency, consistency, machine readability, output quality, latency, token consumption, and cost, Version 3 is the most suitable candidate for deployment. Therefore, it was assigned the production alias in MLflow.

5. Conclusion

This assignment demonstrated how LLMOps practices can be used to manage and optimize large language model applications. By evaluating latency, token usage, cost, structured outputs, and prompt performance, it was possible to identify the most effective prompt configuration. MLflow provided a centralized platform for prompt versioning, evaluation, comparison, and promotion. Among all evaluated versions, Prompt Version 3 delivered the best combination of speed, reliability, JSON compliance, and cost efficiency, making it the preferred choice for production deployment in the EduAssist AI academic question-answering assistant.